

Roteiro de Ambientes — Gestor (DEV x PROD)

Referência: tag `antes-promocao-2026-10-05` · 05/10/2026 Pasta DEV: `C:\Users\mjtam\developer` (branch `dev`) Pasta PROD: `C:\Users\mjtam\developer-prod` (branch `main`)

1. O que foi feito

Superadmin (validado)

- Acesso irrestrito a todas as empresas para `mjtamanhoni@gmail.com` (Go, BFF e frontend).
- Correção do bug de login "Usuário não localizado".

Higiene do Git

Commit	O que fez
<code>9102d4e2</code>	Removeu 13.763 arquivos do versionamento (node_modules, dist, .env) e criou regras de ignore
<code>503f859f</code>	Versionou o que regras gerais bloqueavam por engano: manifests dos mobiles, <code>gradlew.bat</code> , <code>build-apk.bat</code> , ícones Android, migrações <code>.sql</code>
<code>b210fa15</code>	Parametrizou portas via <code>.env</code> (vite, publicar, iniciar_tudo) e criou os scripts de operação

Segurança dos segredos (05/10/2026)

O repositório no GitHub é público. Por isso:

O que	Status
Segredo JWT do PROD	ROTACIONADO — valor novo só nos <code>.env</code> locais do PROD (Go <code>JWT_SECRET</code> = BFF <code>HORSE_JWT_SECRET</code>)
Segredo JWT antigo	invalidado (fica no histórico do Git, mas não vale mais)
<code>horseApi.test.ts</code>	deixou de hardcodar o segredo → lê <code>process.env.HORSE_JWT_SECRET</code>
<code>curso/.../config.ts</code>	fallback trocado por <code>sua_chave_jwt_aqui</code>
<code>.env</code> / keystores	continuam fora do Git (cópia manual)

Consequência da rotação: quem estava logado no PROD precisa logar de novo.

Estrutura de ambientes

- Tag `baseline-superadmin-2026-10-02` — ponto de retorno garantido.
- Branch `dev` para testes; `main` só muda por promoção.
- Clone PROD em `C:\Users\mjtam\developer-prod` (mesmo repositório Git).
- Banco `gestor_dev` criado por cópia do `gestor`.

Separação de configuração (cada pasta tem a sua — não versionada)

Arquivo	DEV (testes)	PROD (produção)
Go <code>src\.env</code>	porta 9001, banco <code>gestor_dev</code> , JWT próprio	porta 9000, banco <code>gestor</code> , JWT próprio
BFF <code>server\.env</code>	porta 3002 → Go 9001	porta 3001 → Go 9000
Client <code>.env.local</code>	Vite 5174 → BFF 3002	não existe (produção serve dist)
Mobiles <code>.env</code>	<code>localhost:9001</code>	<code>localhost:9000</code> / duckdns

JWTs diferentes de propósito: token emitido no DEV **não vale** no PROD.

Cópias manuais para o PROD (não versionadas por segurança)

Keystores de assinatura APK, `local.properties`, `Fotos/`, `apk/`, `data/`.

2. As duas pastas

C:\Users\mjtam\developer ← TESTES/DEV (branch dev) é aqui que você programa
C:\Users\mjtam\developer-prod ← PRODUÇÃO (branch main) NUNCA edite arquivo aqui

	DEV	PROD
Go	:9001	:9000
BFF	:3002	:3001
Frontend	:5174 (Vite, hot reload)	:3001 (dist estático)
Banco	<code>gestor_dev</code>	<code>gestor</code>
Como sobe	<code>iniciar_tudo.bat</code>	<code>publicar.bat</code>

3. Fluxo diário

1. **PROGRAMAR** na pasta DEV (sobe sozinho com hot reload).
2. **TESTAR** em `http://localhost:5174` e rodar os testes antes de commitar:

```
cd FrontEnd\src\server && npm run lint && npm test
cd FrontEnd\src\client && npm run lint && npm test
```

3. **COMMITAR** na pasta DEV, branch `dev`:

```
git add -A
git commit -m "descricao do que mudou"
```

4. QUANDO ESTIVER OK, rodar `aplicar-no-prod.bat` (na pasta DEV).

4. Promover DEV → PROD (`aplicar-no-prod.bat`)

Rodar na pasta DEV. Ele faz sozinho, com segurança:

1. **Recusa** se houver alteração não commitada (nunca promove meio-trabalho).
2. `git fetch . dev:main` → `main` avança para o `dev` (fast-forward, não bagunça sua pasta).
3. Chama o `atualizar.bat` do PROD → `git pull` + rebuild (client, BFF, Go) + restart.
4. **Depois** (fora do script), envie para o GitHub, senão o repositório remoto fica para trás:

```
git push origin main dev --tags
```

Resultado: o PROD roda exatamente o commit que você testou. As janelas novas abrem automaticamente.

5. Rollback (`rollback.bat` — pasta PROD)

1. rodar `rollback.bat`
 2. escolher a tag/commit (ex.: `baseline-superadmin-2026-10-02`)
 3. confirmar com "s"
- `git reset --hard` + rebuild + restart automáticos

Regra prática: crie tag antes de cada promoção importante:

```
git tag antes-promocao-AAAA-MM-DD
```

6. Comandos Git de bolso

Quero...	Comando (pasta DEV)
Ver o que mudei	<code>git status</code> / <code>git diff</code>
Salvar no histórico	<code>git add -A</code> + <code>git commit -m "..."</code>
Commit com confirmação (script)	rodar <code>Commit_Git.bat</code> (§11.0)
Ver histórico	<code>git log --oneline -10</code>
Promover para PROD	rodar <code>aplicar-no-prod.bat</code>
Enviar ao GitHub	<code>git push origin main dev --tags</code>
Marcar um ponto	<code>git tag nome-da-tag</code>
Voltar código antigo (PROD)	rodar <code>rollback.bat</code>
Ver em que commit está o PROD	<code>git -C C:\Users\mjtam\developer-prod log --oneline -1</code>

7. Regras de ouro

1. Programar só na pasta DEV. O PROD só recebe código via `git pull`.
2. Nunca `git commit` na pasta PROD — o pull falharia e o histórico bagunça.
3. `.env` e keystores não estão no Git — nunca versionar; se clonar de novo, copiar à mão.
4. Committe sempre na `dev`; `main` só muda por promoção (`aplicar-no-prod.bat`).
5. Rollback é `git reset` + rebuild — **nunca edite arquivo no PROD** para "consertar" (será sobrescrito no próximo pull).
6. Antes de promoção grande: **tag**. Depois dela, rollback é imediato.
7. Nunca **hardcode segredo em arquivo versionado** (teste, config, doc). O repo é **público** — segredo só em `.env`. Se vaziar: **rotacionar**.
8. Nunca rode `npm install` no PROD (mexe no `package-lock.json` e suja a árvore). Gere o lock na DEV e promova.

8. Estado no momento desta documentação

- DEV = `dev` · PROD = `main` — promoção de 05/10/2026 (tag `antes-promocao-2026-10-05`).
- Árvores Git limpas nas duas pastas (`package-lock.json` do PROD realinhado na DEV).
- Segredo JWT do PROD **rotacionado** (ver §1); Go e BFF conferem em cada ambiente.
- GitHub (`origin`) **sincronizado** (enviado em 05/10/2026): `main`, `dev` e as duas tags. Sempre que promover, feche com `git push origin main dev --tags` (§4, passo 4).
- Backup de 05/10/2026: `projects\Gestor_Backup_2026-10-05_200237\` (Gestor + curso, sem `node_modules`). Os `.env` do PROD antes da rotação estão em `_env_PROD_antes_rotacao\`.

9. Se algo falhar

Sintoma	Causa provável	Solução
Porta ocupada ao subir	processo antigo ainda rodando	feche a janela antiga ou feche via Task Manager
"Há alterações não commitadas" no promoção	trabalho meio pronto na DEV	commite ou <code>git stash</code> e rode de novo
PROD subiu mas dá erro de JWT	<code>.env</code> trocado entre pastas	confira <code>HORSE_JWT_SECRET</code> (BFF) = <code>JWT_SECRET</code> (Go) dentro de cada ambiente
Login falha só no DEV	banco <code>gestor_dev</code> sem dados	é esperado se você criou por cópia; dados mudam só no seu banco de teste
Quer recomeçar o PROD do zero	—	<code>rollback.bat</code> → tag desejada
Promoção para: "Há alterações não commitadas"	<code>package-lock.json</code> mexido no PROD por <code>npm install</code>	no PROD: <code>git checkout -- <arquivo></code> ; gere o lock na DEV
Usuário deslogado depois da promoção	segredo JWT rotacionado (§1)	é esperado — logar de novo

10. Backups locais (antes de mexer em algo grande)

Convenção de nome: `Gestor_Backup_AAAA-MM-DD_HHmss` dentro de `projects\`.

```
$stamp = Get-Date -Format 'yyyy-MM-dd_HHmss'
$dest = "C:\Users\mjtam\developer\projects\Gestor_Backup_$stamp"
robocopy "C:\Users\mjtam\developer\projects\Gestor" "$dest\Gestor" /E /XD node_modules dist
/NFL /NDL /NJH
```

Backup	Conteúdo
<code>Gestor_Backup_2026-09-21_195403</code>	anterior (1.351 arquivos)
<code>Gestor_Backup_2026-10-05_200237</code>	Gestor + curso (7.852 arquivos, sem <code>node_modules</code>) + <code>_env_PROD_antes_rotacao</code>

Backups são ignorados pelo Git (`Gestor_Backup_*` no `.gitignore`) — ficam só no disco.

11. Passo a passo — realizar o commit (DEV e PROD)

Pastas completas (os dois repositórios são independentes):

Ambiente	Repositório (raiz)	Pasta do projeto
DEV	C:\Users\mjtam\developer	C:\Users\mjtam\developer\projects\Gestor
PROD	C:\Users\mjtam\developer-prod	C:\Users\mjtam\developer-prod\projects\Gestor

11.0 Forma rápida — o script `Commit_Git.bat`

Em vez de digitar os comandos, use o script pronto. Ele existe nas **duas** pastas:

Ambiente	Caminho completo
DEV	C:\Users\mjtam\developer\projects\Gestor\Commit_Git.bat
PROD	C:\Users\mjtam\developer-prod\projects\Gestor\Commit_Git.bat

Como usar — duplo clique (ele pergunta tudo) ou já passando o comentário no prompt:

```
cd /d C:\Users\mjtam\developer\projects\Gestor
Commit_Git.bat "mensagem do commit"
```

O que ele faz sozinho:

1. Detecta em qual pasta está e oferece `[1] DEV` / `[2] PROD` (Enter = manter o detectado).
2. Confere que é repositório Git e mostra a **branch** e o `git status --short`.
3. Se a árvore estiver **limpa**, avisa e sai sem fazer nada.
4. No **PROD** mostra o aviso (§11.3) e exige `s` para seguir.
5. Pede o **comentário** (ou usa o argumento) — recusa se vier vazio.
6. Deixa escolher: `[A]` adicionar tudo · `[E]` escolher um a um · `[C]` cancelar.
7. Mostra **o que vai entrar** + o comentário e pede confirmação `[S/n]`.
8. Faz o `git commit` e imprime os 3 últimos commits.
9. Na **DEV**: sugere o `aplicar-no-prod.bat` e pergunta se já envia ao GitHub (`git push`). No **PROD**: lembra de levar a mesma mudança para a DEV (§11.3).

As perguntas aceitam **Enter** para o padrão. Caminhos de arquivo no item 6 são relativos à raiz do repositório (ex.: `projects/Gestor/src/meuarquivo.ts`).

11.1 Commit na DEV (caminho normal — é aqui que você trabalha)

Abra o **Prompt de Comando** (ou PowerShell) e cole:

```
cd /d C:\Users\mjtam\developer
git status
```

Confira o que mudou (`git status` lista em vermelho os arquivos alterados):

```
:: adicionar arquivos específicos (use o caminho completo relativo à raiz)
cd /d C:\Users\mjtam\developer
git add projects/Gestor/src/meuarquivo.ts
git add projects/Gestor/docs/ROTEIRO-AMBIENTES.md

:: ou adicionar tudo de uma vez
git add -A

:: conferir o que entrou no stage
git status

:: gravar no histórico (branch dev)
git commit -m "mensagem clara do que mudou"

:: conferir
git log --oneline -3
```

Cuidados obrigatórios:

Situação	O que fazer
Arquivo novo <code>.json</code> / <code>.sql</code> / <code>.bat</code> / <code>.png</code> / <code>.ps1</code>	o <code>.gitignore</code> tem globais que ignoram esses tipos → use <code>git add -f projects/Gestor/caminho/arquivo.json</code>
Arquivo <code>.env</code>	NUNCA commitar — já está no <code>.gitignore</code> ; segredo só em <code>.env</code>
<code>package-lock.json</code> mexido sem querer	<code>git checkout -- projects/Gestor/Mobile/Cliente/package-lock.json</code>
Quer desfazer o stage	<code>git reset <arquivo></code>

Depois de commitar na DEV, valide (testes/lint) e **promova** com `aplicar-no-prod.bat` (§4).

11.2 No PROD — como os commits chegam (o caminho correto)

No PROD você **NÃO** commita. Ele só recebe o que já foi commitado e testado na DEV:

```
cd /d C:\Users\mjtam\developer-prod
git status
git log --oneline -3
```

- `git status` deve ficar **limpo** (se aparecer algo sujo, é `package-lock.json` : `git checkout -- <arquivo>`).
- O commit **já está lá** porque o `aplicar-no-prod.bat` faz o `git pull` sozinho.
- Se quiser **confirmar** que PROD = DEV, compare os hashes:

```
cd /d C:\Users\mjtam\developer      & git rev-parse HEAD
cd /d C:\Users\mjtam\developer-prod  & git rev-parse HEAD
```

Os dois precisam ser iguais.

11.3 Exceção — commit no PROD (só se realmente for inevitável)

 Quebra a regra de ouro (§7, item 5) e o próximo `aplicar-no-prod.bat` vai dar conflito. Só faça isso se estiver com o PROD quebrado e sem como levar a correção pela DEV.

```
cd /d C:\Users\mjtam\developer-prod
git status
git add projects/Gestor/caminho/arquivo
git commit -m "CORRECAO EMERGENCIAL: ..."
git log --oneline -3
```

Obrigatório em seguida — senão a próxima promoção trava:

1. Copie o mesmo arquivo para a DEV (o caminho abaixo é literal):

```
copiar: C:\Users\mjtam\developer-prod\projects\Gestor\...\arquivo
para : C:\Users\mjtam\developer\projects\Gestor\...\arquivo
```

2. Commit normalmente na DEV (§11.1): `git add` + `git commit`.
3. Faça o push dos dois (`git push origin main` e `git push origin dev`).
4. Na próxima promoção o conteúdo já é idêntico → o `git pull` resolve sozinho.

Alternativa mais simples (recomendada): desfça o commit do PROD, corrija na DEV e promova de novo:

```
cd /d C:\Users\mjtam\developer-prod
git reset --hard HEAD~1
git status
:: aí corrige na DEV e roda aplicar-no-prod.bat
```